

Ex.no:5b	DATA VISUALIZATION USING MATPLOTLIB (CAR PREDICTION DATA)
Date:	

Aim: To visualize dataset features and distributions of the Car Prediction dataset using line charts, bar charts, pie charts, histograms, scatter plots, and box plots using Matplotlib.

ALGORITHM 1: LINE CHART

Objective: To visualize the yearly average selling price trend.

Steps

1. Load the car prediction dataset.
2. Group data by Year and calculate the average Selling_Price.
3. Plot the yearly average selling price using a line chart.
4. Add title and axis labels.
5. Display the graph.

Interpretation: The line chart shows how the average selling price of cars varies across different manufacturing years, helping identify pricing trends over time.

```
# Program 1: Line Chart
yearly_sales = df.groupby('Year')['Selling_Price'].mean()
plt.plot(yearly_sales.index, yearly_sales.values, marker='o')
plt.title("Average Selling Price by Year")
plt.xlabel("Year")
plt.ylabel("Selling Price (in Lakhs)")
plt.show()
```

ALGORITHM 2: BAR CHART

Objective: To compare total selling price across different fuel types.

Steps

1. Select the Fuel_Type and Selling_Price columns.

2. Group by Fuel_Type and compute the total selling price.
3. Create a bar chart using the fuel type sales values.
4. Add title and axis labels.
5. Display the chart.

Interpretation: The bar chart helps compare cumulative selling price values across Petrol, Diesel, and CNG fuel types.

```
# Program 2: Bar Chart
fuel_sales = df.groupby('Fuel_Type')['Selling_Price'].sum()
plt.bar(fuel_sales.index, fuel_sales.values)
plt.title("Total Selling Price by Fuel Type")
plt.xlabel("Fuel Type")
plt.ylabel("Total Selling Price (in Lakhs)")
plt.show()
```

ALGORITHM 3: PIE CHART

Objective: To visualize the distribution of fuel types among vehicles.

Steps

1. Select the Fuel_Type column and count values.
2. Create a pie chart using the fuel type frequencies.
3. Display percentage labels.
4. Add a suitable title.
5. Display the chart.

Interpretation: The pie chart shows the percentage share of each fuel type (Petrol, Diesel, CNG) within the dataset.

```
# Program 3: Pie Chart
fuel_counts = df['Fuel_Type'].value_counts()
plt.pie(fuel_counts.values, labels=fuel_counts.index, autopct="%1.1f%%")
plt.title("Fuel Type Distribution")
plt.show()
```

ALGORITHM 4: HISTOGRAM

Objective: To visualize the distribution of car selling prices.

Steps

1. Select the Selling_Price column.
2. Divide the selling price values into bins.
3. Create a histogram.
4. Add title and axis labels.
5. Display the histogram.

Interpretation: The histogram shows the frequency distribution of car selling prices across different price intervals.

```
# Program 4: Histogram
plt.hist(df['Selling_Price'], bins=15)
plt.title("Selling Price Distribution")
plt.xlabel("Selling Price (in Lakhs)")
plt.ylabel("Frequency")
plt.show()
```

ALGORITHM 5: SCATTER PLOT

Objective: To visualize the relationship between Present Price and Selling Price.

Steps

1. Select Present_Price as the X-axis variable.
2. Select Selling_Price as the Y-axis variable.
3. Create a scatter plot.
4. Add title and axis labels.
5. Display the graph.

Interpretation: The scatter plot shows the correlation and variation between showroom present price and resale selling price.

```
# Program 5: Scatter Plot
plt.scatter(df['Present_Price'], df['Selling_Price'])
plt.title("Present Price vs Selling Price")
plt.xlabel("Present Price (in Lakhs)")
```

```
plt.ylabel("Selling Price (in Lakhs)")  
plt.show()
```

ALGORITHM 6: BOX PLOT

Objective: To visualize the distribution and detect outliers in selling prices.

Steps

1. Select the Selling_Price column.
2. Create a box plot.
3. Add a suitable title and axis label.
4. Display the plot.

Interpretation: The box plot summarizes the distribution, median, quartiles, and identifies potential outliers in selling prices.

```
# Program 6: Box Plot  
plt.boxplot(df['Selling_Price'])  
plt.title("Selling Price Box Plot")  
plt.ylabel("Selling Price (in Lakhs)")  
plt.show()
```

ALGORITHM 7: MULTIPLE LINE CHARTS

Objective: To compare average Selling Price and Present Price across manufacturing years.

Steps

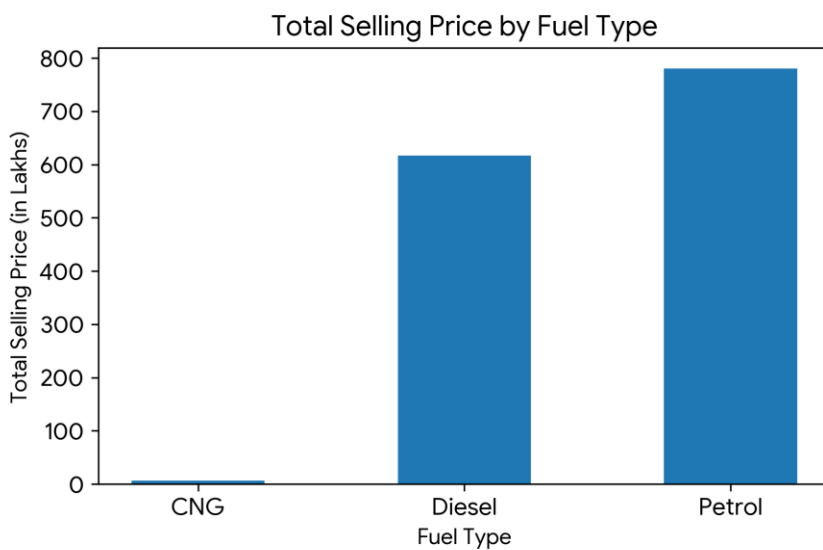
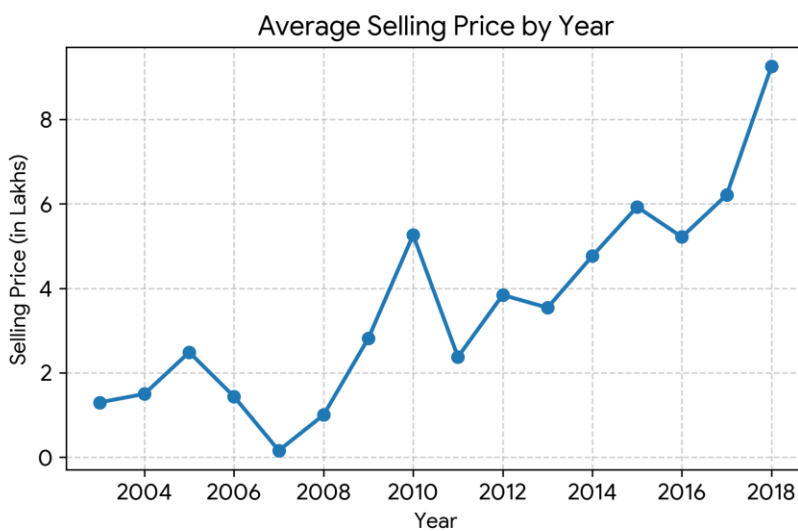
1. Group data by Year and calculate mean Selling_Price and Present_Price.
2. Plot average Selling_Price using a line chart.
3. Plot average Present_Price using another line chart.
4. Add title, axis labels, and legend.
5. Display the graph.

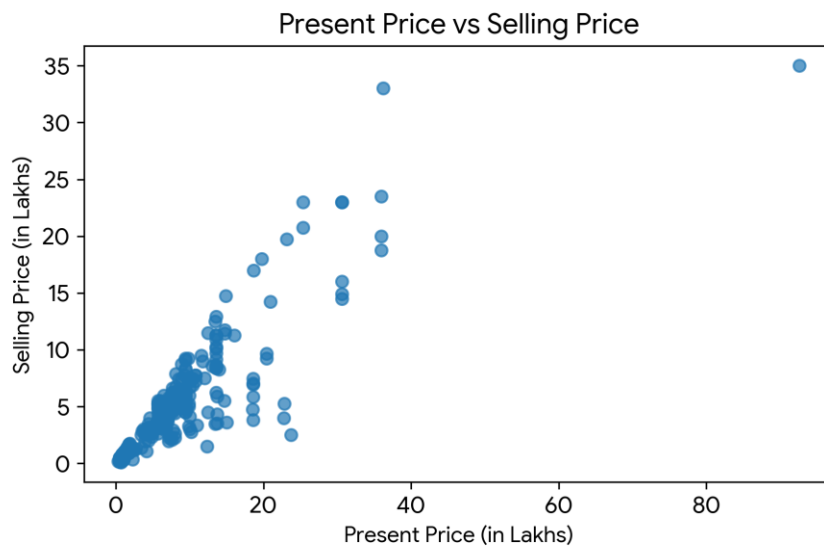
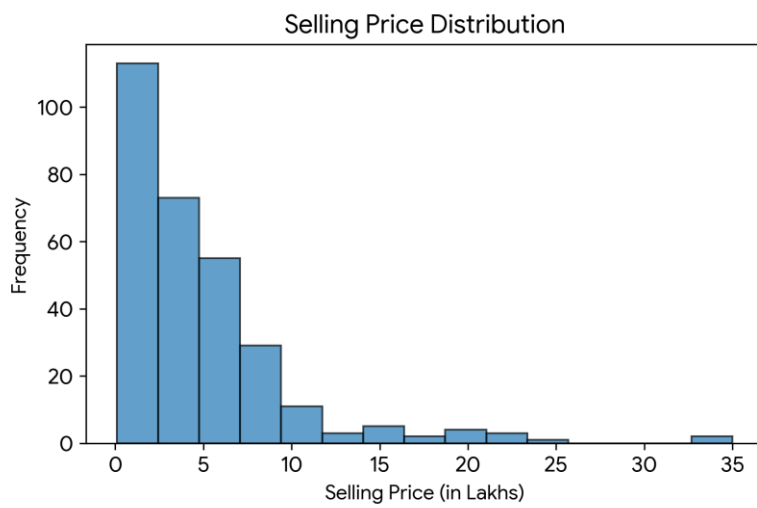
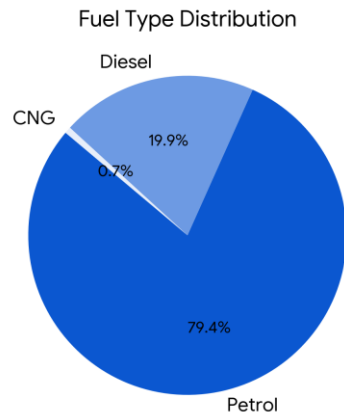
Interpretation: The multiple line chart compares average original present prices and resale selling prices across different vehicle manufacturing years.

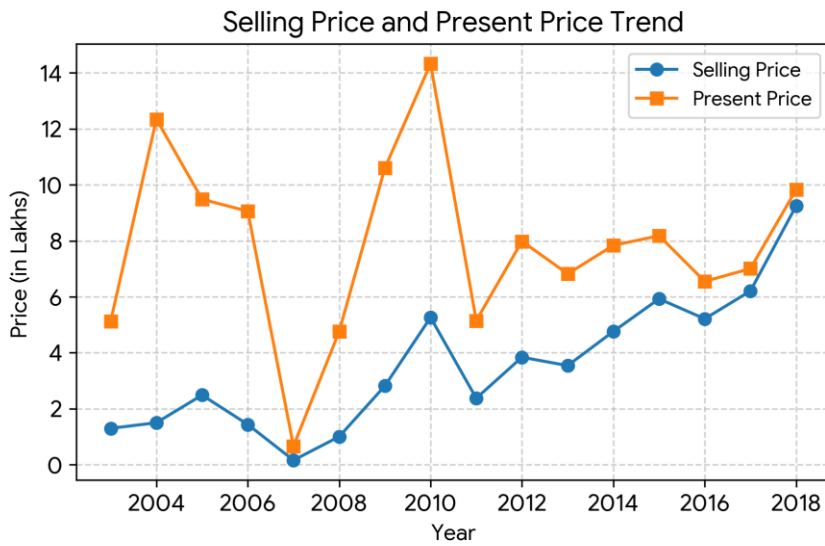
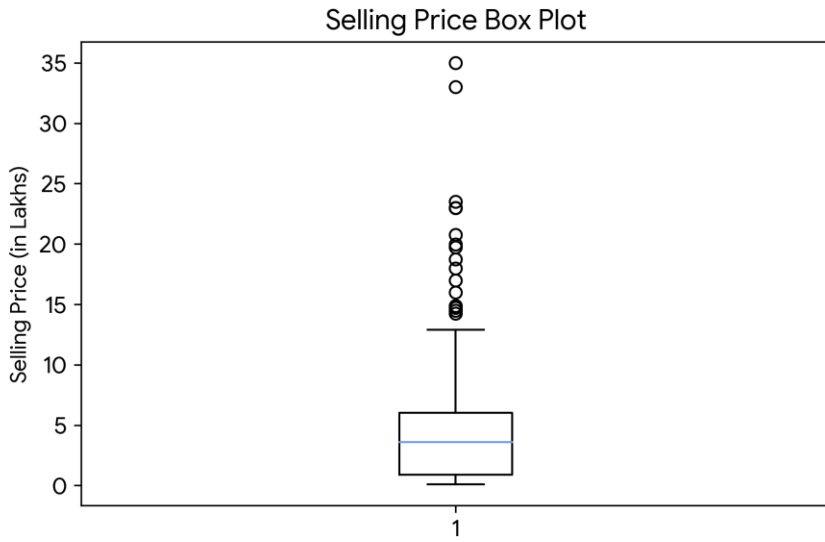
```
# Program 7: Multiple Line Charts  
yearly_prices = df.groupby('Year')[['Selling_Price', 'Present_Price']].mean()  
plt.plot(yearly_prices.index, yearly_prices['Selling_Price'], label="Selling
```

```
Price")
plt.plot(yearly_prices.index, yearly_prices['Present_Price'], label="Present
Price")
plt.title("Selling Price and Present Price Trend")
plt.xlabel("Year")
plt.ylabel("Price (in Lakhs)")
plt.legend()
plt.show()
```

Output:







Result: